

C++ 연습 문제: 14. 가변 인자 템플릿, 완벽한 전달과 파일 입출력

1. 가변 인자 템플릿(Variadic Templates)에 대한 설명으로 올바른 것은? (하)

1. 개수가 고정된 단 하나의 타입만 매개변수로 받을 수 있다.
2. 0개 이상의 서로 다른 타입의 인자들을 템플릿 매개변수로 받아 처리할 수 있다.
3. 반드시 실행 시점에만 인자의 개수와 타입이 결정된다.
4. 클래스 템플릿에는 적용할 수 없고 오직 함수 템플릿에만 사용할 수 있다.

2. 가변 인자 템플릿에서 재귀 호출을 통해 인자들을 처리할 때, 남은 인자가 없을 때 호출을 멈추기 위해 작성하는 일반적인 기준 함수를 무엇이라고 부르는가? (중)

1. 종료 함수 (Base Case)
2. 가상 함수 (Virtual Function)
3. 소멸자 함수 (Destructor)
4. 친구 함수 (Friend Function)

3. C++17에서 도입된 기능으로, 컴파일 시점에 가변 인자 템플릿의 인자들을 쉼표 연산자 등과 함께 순회하며 코드를 자동으로 펼쳐내는 문법은 무엇인가? (상)

1. 폴드 표현식 (Fold Expression)
2. 람다 표현식 (Lambda Expression)
3. 매크로 확장 (Macro Expansion)
4. 타입 캐스팅 (Type Casting)

4. 함수 템플릿에서 인자의 좌측값(lvalue)과 우측값(rvalue) 성질을 그대로 보존하면서 다른 함수로 전달하는 기법을 무엇이라고 하는가? (하)

1. 얕은 복사 (Shallow Copy)
2. 완벽한 전달 (Perfect Forwarding)
3. 동적 바인딩 (Dynamic Binding)
4. 예외 가로채기 (Exception Catching)

5. 완벽한 전달(Perfect Forwarding)을 구현할 때 함수 매개변수 타입으로 사용되며, 템플릿 타입 매개변수 T에 대해 좌측값과 우측값을 모두 받아내는 보편 참조(Forwarding Reference)의 올바른 문법 형태는 무엇인가? (중)

1. const T&
2. T*
3. T&& (단, T는 템플릿 타입 매개변수)
4. const T&&

6. 템플릿 매개변수 타입이 T&& 형태로 선언되었을 때, 이것이 항상 우측값 참조가 되지 않고 좌측값과 우측값을 모두 받아내는 보편 참조로 동작하도록 만드는 C++의 핵심 규칙은 무엇인가? (상)

1. 참조 축약 규칙 (Reference Collapsing Rules)
2. 가상 함수 오버라이딩 규칙 (Virtual Overriding Rules)
3. 다중 상속 모호성 규칙 (Multiple Inheritance Rules)
4. 템플릿 인스턴스화 지연 규칙 (Late Instantiation Rules)

7. 완벽한 전달을 수행할 때 인자를 원래의 성질(좌/우측값) 그대로 유지하며 다른 함수로 캐스팅하여 전달하기 위해 사용하는 표준 라이브러리 함수는 무엇인가? (중)

1. `std::move`
2. `std::forward`
3. `std::copy`
4. `std::swap`

8. `std::forward<T>(arg)`와 `std::move(arg)`의 주된 차이점으로 가장 올바른 것은 무엇인가? (상)

1. `std::forward`는 항상 우측값으로 바꾸고, `std::move`는 항상 좌측값으로 바꾼다.
2. `std::forward`는 인자의 원래 성질(좌측값/우측값)을 조건에 맞게 보존하여 전달하고, `std::move`는 무조건 우측값으로 캐스팅한다.
3. `std::forward`는 클래스 내부에서만 쓸 수 있고, `std::move`는 전역에서만 쓸 수 있다.
4. 두 함수는 완전히 동일한 동작을 수행하는 별칭이다.

9. 컴파일 시점에 특정 식이나 표현식이 예외를 발생시키지 않는지 판별하여 부울 값을 반환하는 연산자는 무엇인가? (중)

1. `sizeof`
2. `noexcept`
3. `alignof`
4. `typeid`

10. 가변 인자 템플릿에서 전달된 인자의 총 개수를 컴파일 시점에 상수(정수)로 반환해주는 연산자는 무엇인가? (하)

1. `sizeof...`
2. `lengthof`
3. `countof`
4. `argcount`

11. 템플릿 메타프로그래밍이나 복잡한 객체 생성 과정에서, 객체의 실체를 실제로 만들지 않고도 특정 타입의 멤버 함수나 표현식의 타입을 추론할 수 있게 가짜 객체를 만들어 반환하는 표준 유틸리티 함수는 무엇인가? (중)

1. `std::forward`
2. `std::declval`
3. `std::move`
4. `std::cref`

12. C++에서 파일 입출력을 수행하기 위해 파일을 읽고 쓰기 모두 지원하며 `<fstream>` 헤더에 정의된 대표적인 스트림 클래스는 무엇인가? (하)

1. `std::ifstream`
2. `std::ofstream`
3. `std::fstream`
4. `std::iostream`

13. 파일 스트림 객체를 생성하여 파일을 열었을 때, 파일 열기에 실패했는지 여부를 확인하는 방법으로 가장 적절하지 않은 것은 무엇인가? (중)

1. `if (!fileStream)` 형태의 불리언 변환 검사
2. `fileStream.is_open()` 함수의 반환값이 `false`인지 검사
3. `fileStream.bad()` 함수만 단독으로 호출하여 파일 열기 실패를 검사
4. `if (fileStream.fail())` 조건문 검사

14. 파일 스트림을 사용할 때, 파일을 다 사용한 후 명시적으로 파일을 닫고 자원을 해제하기 위해 호출하는 멤버 함수는 무엇인가? (하)

1. `close()`
2. `finish()`
3. `terminate()`
4. `release()`

15. 파일 스트림에서 텍스트 기반 스트림이 아닌 바이트 단위의 원시 데이터를 손실 없이 그대로 읽거나 쓸 때 주로 사용하는 멤버 함수 조합은 무엇인가? (중)

1. `getline`과 `putline`
2. `read`와 `write`
3. `load`와 `save`
4. `input`과 `output`

16. 표준 입출력 스트림에서 버퍼에 쌓인 출력 내용을 강제로 실제 기기나 파일로 즉시 밀어내기 위해 사용하는 조작자는 무엇인가? (중)

1. `std::endl` (또는 명시적인 `std::flush`)
2. `std::reset`
3. `std::clear`
4. `std::sync`

17. 스마트 포인터 `std::unique_ptr`를 함수 인자로 전달할 때, 복사가 불가능하므로 소유권을 안전하게 함수 내부로 넘겨주기 위해 호출해야 하는 함수는 무엇인가? (중)

1. `std::move`
2. `std::forward`
3. `std::ref`

4. `std::copy`

18. 컨테이너에 새로운 원소를 추가할 때, 불필요한 임시 객체의 복사나 이동을 없애고 컨테이너 내부 메모리에서 객체를 직접 생성하기 위해 사용하는 대표적인 멤버 함수는 무엇인가? (상)

1. `push_back`
2. `insert`
3. `emplace_back`
4. `append`

19. 완벽한 전달을 활용하여 객체를 생성하거나 전달하는 공장(Factory) 함수를 설계할 때, 매개변수 형식을 지정하는 올바른 표준 관행은 무엇인가? (상)

1. `template <typename... Args> void Create(Args... args)`
2. `template <typename... Args> void Create(Args&&... args)`
3. `template <typename... Args> void Create(const Args&... args)`
4. `template <typename... Args> void Create(Args*... args)`

20. C++ 프로그램에서 예외 처리를 수행할 때, 예외가 발생할 가능성이 있는 코드를 감싸는 블록과 예외를 포착하는 블록의 올바른 키워드 조합은 무엇인가? (하)

1. `check`와 `catch`
2. `try`와 `catch`
3. `begin`과 `handle`
4. `throw`와 `except`